PYECHARTS

Sylvie

PYECHARTS

主要内容

1.安装
2.主题河流图
3.树图
4.词云图
5.地理图表
6.3D地图
7.仪表盘和水球图

课程大纲

主题河流图	树图	词云图	地理图表	水球图和仪表盘
伪数据+真实数据实例	普通树图+矩形树图	转化数据并计算频次+普通词云图+ 钻石词云图	地理图标+地图+3D地图	动态水球图+动态仪表盘

Pyecharts简介

Pyecharts 是一个用于生成 Echarts 图表的类库。Echarts 是百度开源的一个数据可视化 JS 库。用 Echarts 生成的图可视化效果非常棒，为了与 Python 进行对接，方便在 Python 中直接使用数据生成图

1.安装

安装Pyecharts时记得关掉所有的VPN

- 安装命令：pip install pyecharts

- 卸载命令：pip uninstall pyecharts

前期准备工作需要导入的所有包

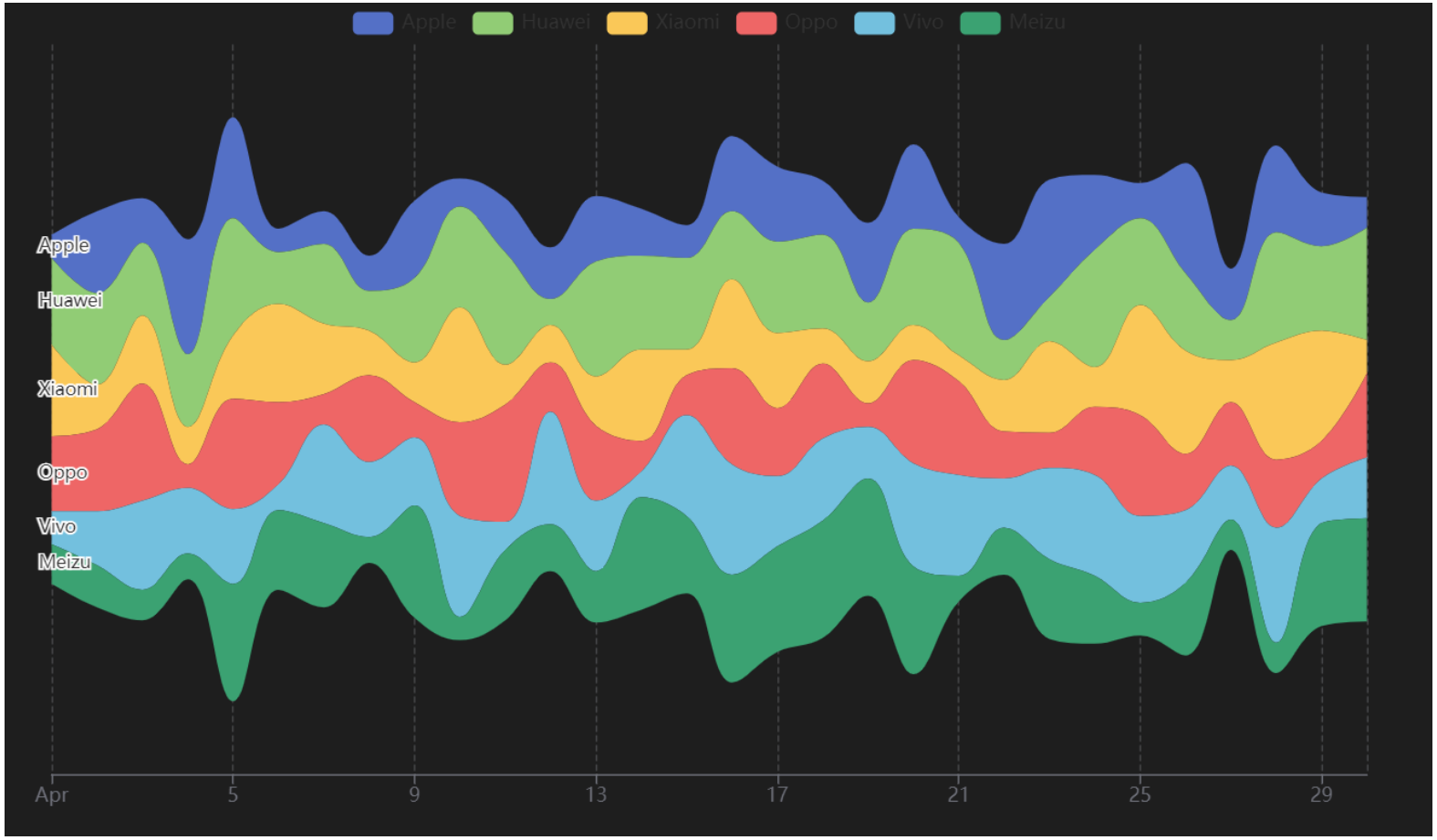
- import chart_studio.plotly as py
- import cufflinks as cf
- cf.set_config_file(offline=True)
- from pyecharts.charts import Bar,WordCloud,ThemeRiver,Geo, Graph
- from pyecharts import options as opts
- from pyecharts.globals import SymbolType,ChartType
- from pyecharts.faker import Faker
- import networkx as nx
- import pandas as pd
- import numpy as np
- import random
- import json

2.主题河流图

2.1用伪数据作的主题河流图

```
# 虚假数据
cate = ['Apple', 'Huawei', 'Xiaomi', 'Oppo', 'Vivo', 'Meizu']
date_list = ["2020/4/{}".format(i + 1) for i in range(30)]
data = [[day, random.randint(10, 50), c] for day in date_list for c in cate]
river = (
 ThemeRiver()
  .add(
    series_name=cate,
    data=data,
    singleaxis_opts=opts.SingleAxisOpts(type_="time")
  )
)

river.render_notebook()
```

用伪数据作的主题河流图



2.2用真实数据作的主题河流图

- 以iOS app近三个月收入数据 （2022年12月18日-2023年3月17日）为例：
- #获取数据（以下数据来源自七喜数据网）
- #目前选定了十大排行榜里的三大app分别是王者荣耀 抖音 和平精英
- #导入数据
- #因为utf可能无法被识别，这里我们用gbk进行编码
- data_wz = pd.read_csv('data/202312-202303/wangzhe.csv',encoding='gbk')
- data_wz["class"] = ["王者荣耀"]*len(data_wz)
- data_dy = pd.read_csv("data/202312-202303/douyin.csv",encoding='gbk')
- data_dy["class"] = ["抖音"]*len(data_dy)
- data_hpjy = pd.read_csv("data/202312-202303/hepingjingying.csv",encoding='gbk')
- data_hpjy["class"] = ["和平精英"]*len(data_hpjy)

用真实数据作的主题河流图

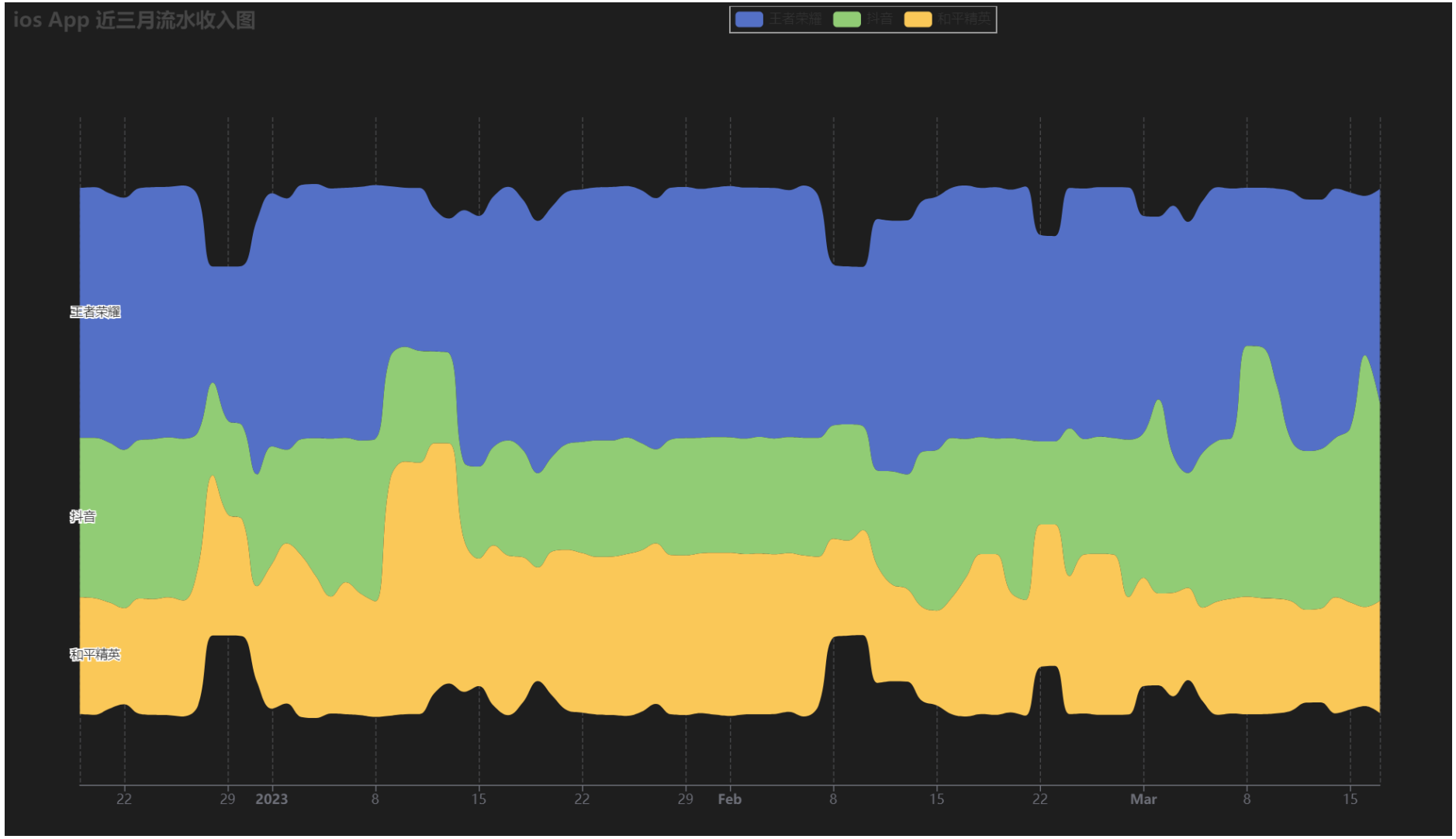
- 以iOS app近三个月收入数据 （2022年12月18日-2023年3月17日）为例：
- #整理相关的数据类型
- series = ["王者荣耀","抖音","和平精英"]
- data = pd.concat([data_wz[:-1],data_dy[:-1],data_hpjy[:-1]])
- data = data.values.tolist()
- data[:5]

```
[['2023/3/17', 3135489, '王者荣耀'],
 ['2023/3/16', 2337371, '王者荣耀'],
 ['2023/3/15', 3458701, '王者荣耀'],
 ['2023/3/14', 3656696, '王者荣耀'],
 ['2023/3/13', 3661757, '王者荣耀']]
```

处理完的数据第一列是时间，第二列是预估收入流水，第三列是app名称

```
#绘制时间河流图
c =(
    ThemeRiver(init_opts=opts.InitOpts(width="1300px", height="800px"))
    .add(
        series_name=series,
        data=data,
        singleaxis_opts=opts.SingleAxisOpts(
            pos_top="100", pos_bottom="100", type_="time"
        ),
    )
    .set_global_opts(
        tooltip_opts=opts.TooltipOpts(trigger="axis", axis_pointer_type="line"),
        legend_opts=opts.LegendOpts(pos_left='50%'),
        title_opts=opts.TitleOpts(title="ios App 近三月流水收入图")
    )
    #.set_global_opts()
    #.render("game_income_time.html"),
)
c.render_notebook()
```

用真实数据作的主题河流图



3.树图

3.1用伪数据作的树图

```
# 虚假数据
data = [
    {"name": "湖南",
     "children": [
            {"name": "长沙",
             "children": [
                {"name": "雨花区", "value": 55},
                {"name": "岳麓区", "value": 34},
                {"name": "天心区", "value": 144},
             ]},
```

用伪数据作的树图

```
# 虚假数据
            {"name": "常德",
             "children": [
                    {"name": "武陵区", "value": 156},
                    {"name": "鼎城区", "value": 134},
             ]},
            {"name": "湘潭", "value": 87},
            {"name": "株洲", "value": 23},
    ],
    }
]
tree = (
    Tree()
    .add("", data)
)
tree.render_notebook()
```

3.2用伪数据作的矩形树图

```
# 虚假数据
data = [
    {"name": "湖南",
     "children": [
            {"name": "长沙",
             "children": [
                {"name": "雨花区", "value": 55},
                {"name": "岳麓区", "value": 34},
                {"name": "天心区", "value": 144},
             ]},
            {"name": "常德",
             "children": [
                    {"name": "武陵区", "value": 156},
                    {"name": "鼎城区", "value": 134},
             ]},
```

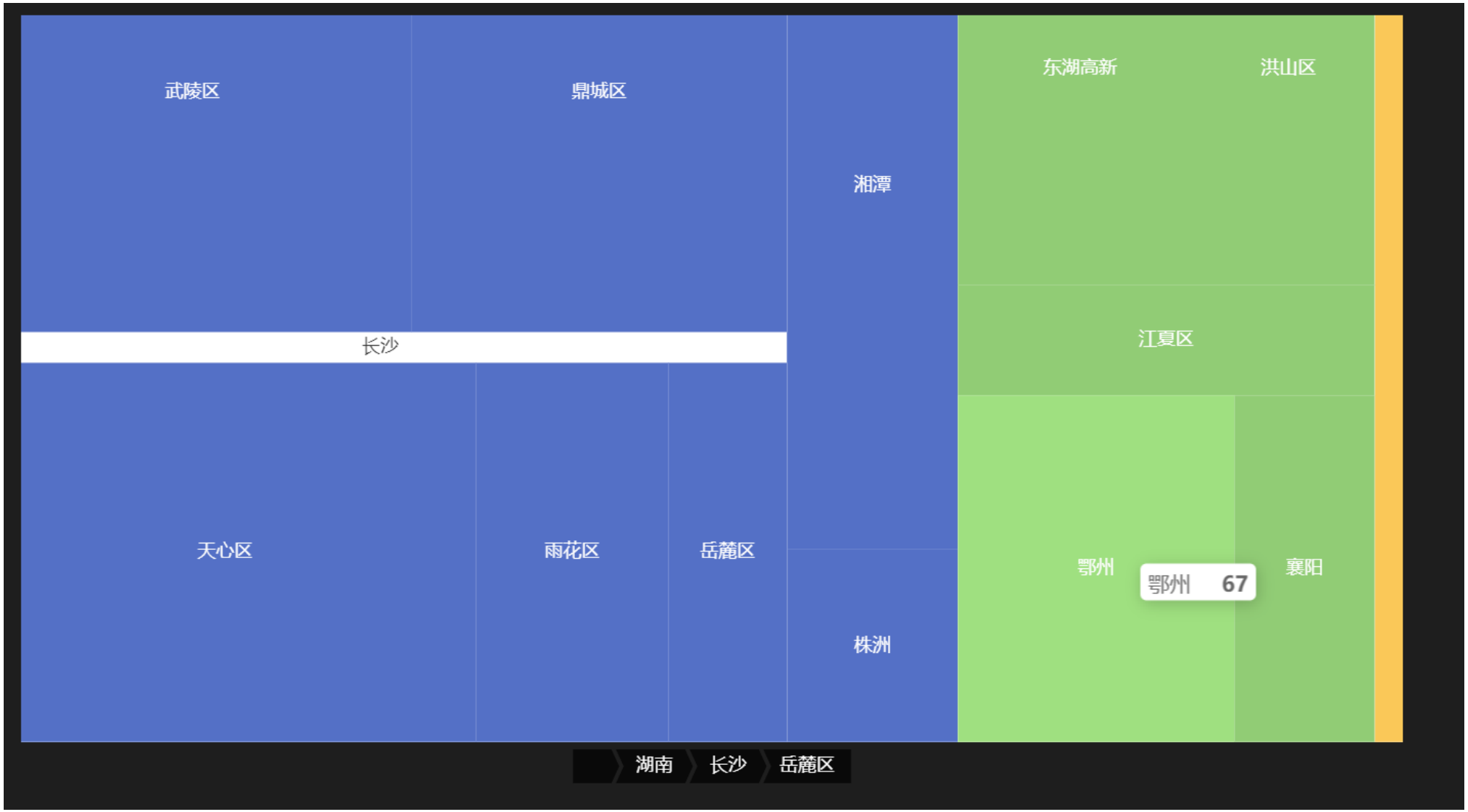
用伪数据作的矩形树图

- # 虚假数据
- {"name": "湘潭", "value": 87},
- {"name": "株洲", "value": 23},
-],
- },
- {"name": "湖北",
- "children": [
- {"name": "武汉",
- "children": [
- {"name": "洪山区", "value": 55},
- {"name": "东湖高新", "value": 78},
- {"name": "江夏区", "value": 34},
-]},
-

用伪数据作的矩形树图

```
# 虚假数据
            {"name": "鄂州", "value": 67},
            {"name": "襄阳", "value": 34},
      ],
      },
    {"name": "北京", "value": 235}
]

treemap = (
    TreeMap()
    .add("", data)
)
treemap.render_notebook()
```

用伪数据作的矩形树图



4.词云图

4.1用伪数据作的词云图

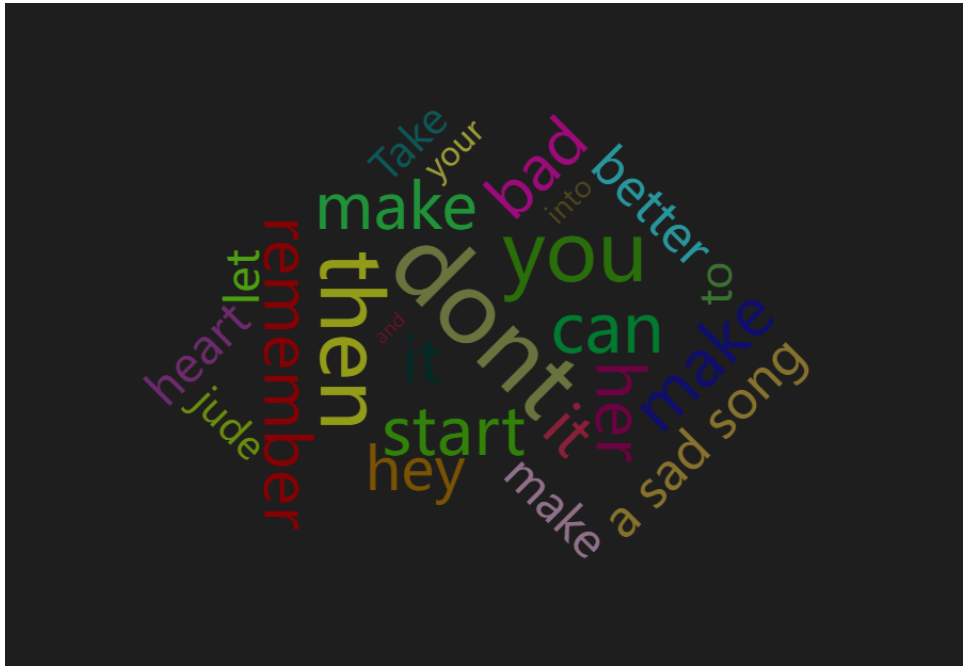
- # 虚假数据
- #数出词的出现的频次，频次越大字母越大
- words = [
- ("hey", 230),
- ("jude", 124),
- ("dont", 436),
- ("make", 255),
- ("it", 247),
- ("bad", 244),
- ("Take", 138),
- ("a sad song", 184),
- ("and", 12),
- ("make", 165),
- ("it", 247),
- ("better", 182),
- ("remember", 255),

用伪数据作的词云图

- # 虚假数据
- ("to", 150),
- ("let", 162),
- ("her", 266),
- ("into", 60),
- ("your", 82),
- ("heart", 173),
- ("then", 365),
- ("you", 360),
- ("can", 282),
- ("start", 273),
- ("make", 265),
-]

用伪数据作的词云图

- # 虚假数据
- wc = (
- WordCloud()
- .add("", words)
-)
- wc.render_notebook()



4.2用真实数据作的词云图

- #我选取了一篇自己用过的论文，一篇有关于创建领域知识图谱的论文，我先利用网页上的pdf转txt进行转换，然后用jieba分词提取关键词词频，最后利用词频来绘制词云图。
- 生产了一个钻石图案的词云图，从图中可以看出最大的两个单词‘knowledge’和'entity’,就是这篇论文里面的常用专业术语。知识图谱中的知识就是knowledge，实体叫做entity。这个词云图很好地展现了这篇论文的主要内容。
- #先把作图要用的包都导入进来
- # WordCloud
- # 论文文本词云图
- import re
- import string
- import jieba
- import jieba.analyse
- import html
- from numpy import *

用真实数据作的词云图

```
txt_path = "data/construction.txt"
with open(txt_path,"rb") as x:
    content = x.read()
    # 正则过滤
    content = re.sub("[{}]+".format(string.punctuation), " ", content.decode("utf-8"))
    # html 转义符实体化
    content = html.unescape(content)
    # 切割
    seg = [i for i in jieba.cut(content, cut_all=True) if i != '']
    # 提取关键词（TF-IDF）
    keywords = jieba.analyse.extract_tags("|".join(seg),
    topK=20, withWeight=True)
```

用真实数据作的词云图

- # 分词与关键词提取
- keywords_weight = keywords
- #with open('./keywords0.txt', 'w') as k0:
- # k0.write(str(keywords0))
- # k0.close()
- print('完成文章关键字提取！')

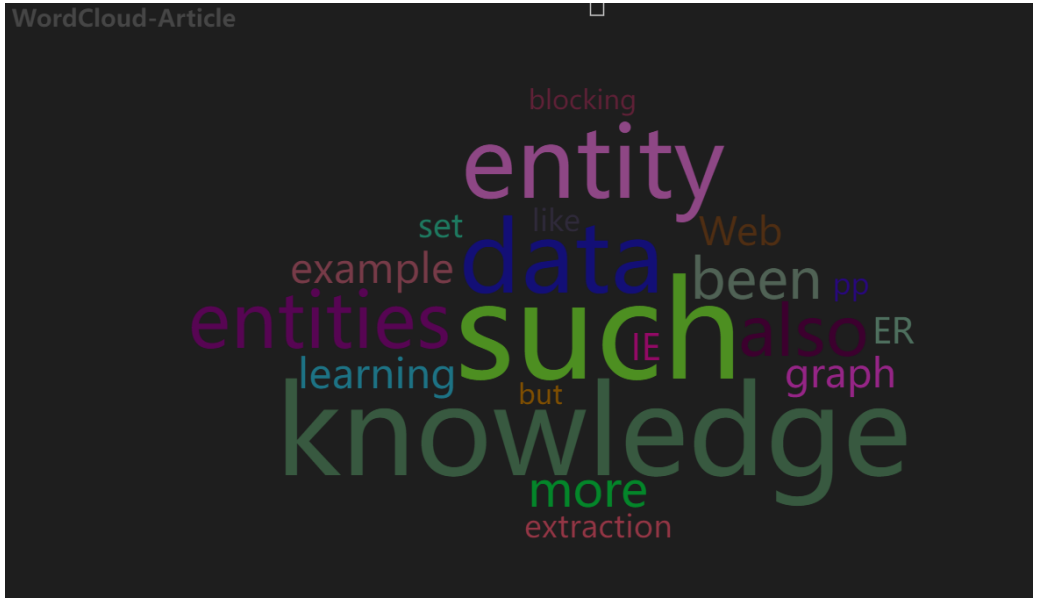
```
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\lychee\AppData\Local\Temp\jieba.cache
Loading model cost 0.536 seconds.
Prefix dict has been built successfully.

完成文章关键字提取!
```

用真实数据作的词云图

```
# 绘制词云图
#以绘制钻石词云图为例
## diamond shape
c = (
    WordCloud()
    .add("", keywords_weight, word_size_range=[20, 100], shape=SymbolType.DIAMOND)
    .set_global_opts(title_opts=opts.TitleOpts(title="WordCloud-Article"))
    #.render("wordcloud_diamond.html")
)
c.render_notebook()
```

用真实数据作的词云图

- #也可以用wordart网站绘制词云图
- 用wordart网站绘制了在线词云图，数据和上面使用的是一样的txt文件，直接将txt文件输入网站中，设置图片样式为你喜欢的图片形式，设置图片中文字的颜色和大小即可，生成对应的图片(Word.Art.png)
- 用wordart网站制作更加地方便和精美



5.地理图表

5.1用伪数据作的地理图表

- #导入所需要的包:
- from pyecharts.charts import *
- from pyecharts.components import Table
- from pyecharts import options as opts
- from pyecharts.commons.utils import JsCode
- import random
- import datetime

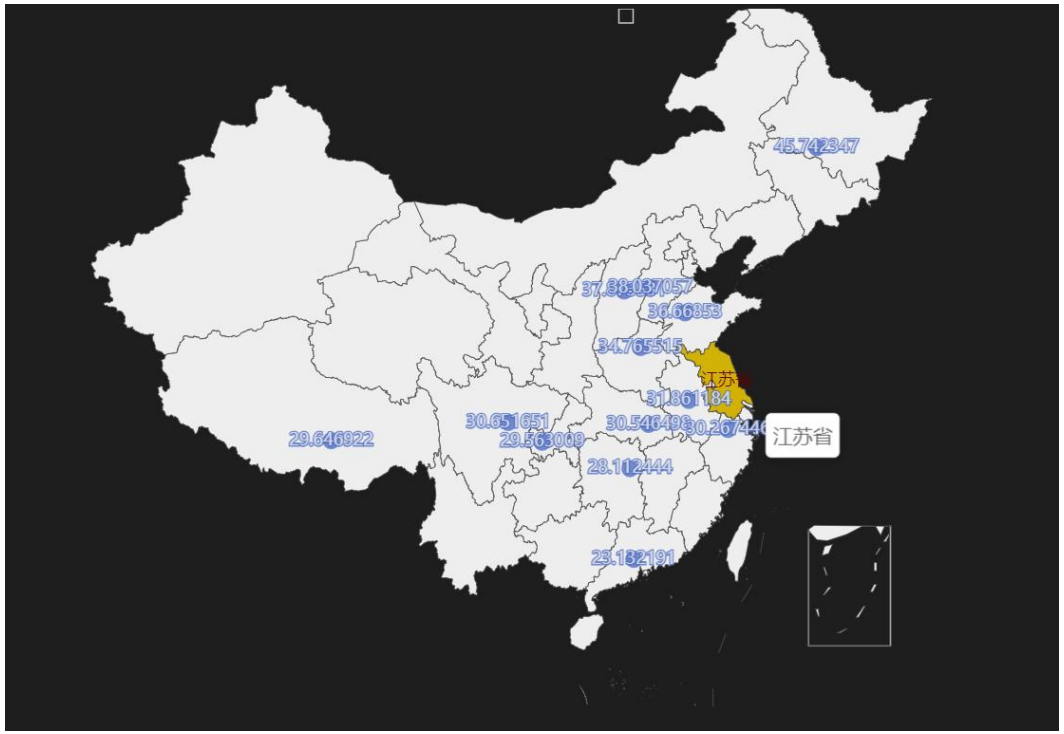
- #用伪数据创建地理坐标系
- # 虚假数据
- province = [
- 　　'广东',
- 　　'湖北',
- 　　'湖南',
- 　　'四川',

用伪数据作的地理图表

- #用伪数据创建地理坐标系
- # 虚假数据
- '四川',
- 　　'重庆',
- 　　'黑龙江',
- 　　'浙江',
- 　　'山西',
- 　　'河北',
- 　　'安徽',
- 　　'河南',
- 　　'山东',
- 　　'西藏']
- data = [(i, random.randint(50, 150)) for i in province]

用伪数据作的地理图表

- #用伪数据创建地理坐标系
- geo = (
- Geo()
- .add_schema(maptype="china")
- .add("", data)
-)
- geo.render_notebook()



5.2用伪数据作的地图

- #用伪数据创建地图
- # 虚假数据
- province = [
- 　　'广东',
- 　　'湖北',
- 　　'湖南',
- 　　'四川',
- 　　'重庆',
- 　　'黑龙江',
- 　　'浙江',
- 　　'山西',
- 　　'河北',
- 　　'安徽',
- 　　'河南',
- 　　'山东',
- 　　'西藏']

用伪数据作的地图

- #用伪数据创建地图
- # 虚假数据
- data = [(i, random.randint(50, 150)) for i in province]
- map_ = (
- Map()
- .add("", data, 'china')
-)
- map_.render_notebook()

谢谢